

Food Delivery App -- Feast-Hut-Kitchen --

Introduction:

Flow Of Work:

Frontend

- Make Three Folders:
 - Backend
 - Frontend
 - Admin
- Cd frontend
 - bun install create vite@latest
 - bun install react-router-dom
 - nun run dev

Remove codes of index.css and app.css

Adjust the app.js code to basic format, not the already defined one.

Download ES7 extension

```
Index.css --
*{
  padding: 0;
  margin: 0;
  box-sizing: border-box;
  font-family: Outfit;
  scroll-behavior: smooth;
}

body {
  min-height: 100vh;
}
```

- Replace names in the index.html file.
- Src --> Components -->
 - Navbar folder
 - Pages -> Home, Menu, Mobile-App, Contact-Us
 - Css property of each ... hover properties
 - Active state logic on clicking the active navbar
 - Dynamic css is also used.
 - Routing in the app.jsx using Browser Router
 - Import all the pages in app.js using routes

Header.jsx in components and import it in the Home.jsx page under pages section

In header add the img and the text on it using proper css

Now

- ExporeMenu.jsx -- A slider that gonna allow to show the menulist under it, imported from the assests folder. Ex burger, cake, veg, maggies etc. and do add the css accordingly.
- Now we goona need to make the Context under src. This would help to store the data and functionality better. Don't have to import data in every file just use useContext and stop again and again fetcing or re-rendering of the components and files
- Make FoodDisplay – shows all the items using map

- Make FoodList – this is passed in above map to show the foodlist in a proper format. Like text, names, item price.
- Add proper CSS for both of the following.
- Do add that transition effect – fadein on each refresh!!
- Add functionality like addCart, RemoveCart, etc in the StoreContext
- Add the footer folder under components and do css and react code for it.
- Make AppDownload under components and do add this function above the footer component in the app.js folder!

Now LoginPop-Up under Components

- Use useState() to have the updation of the logged in or not!
- Make the login popup effect, position
- Make the cart page !! – add functionality from the first page to the cart page and keeping record what data is filled over there and here!
- Do add the addTotalAmount in storeContext and show it to the Cart webpage !!
- Make PlaceOrder.jsx page

Backend

Basics codes –

- Bun init
- Make server.js file
- Bun I install express mongoose jsonwebtoken bcrypt cors dotenv body-parser multer stripe/razorpay validator nodemon

Core Packages

1. **express**
→ Framework to create the backend server and APIs easily.
◆ Example: app.get('/api/users', ...)
2. **mongoose**
→ Used to connect and interact with **MongoDB** (your database).
◆ Helps define schemas and models for data (like users, products, etc).

Authentication & Security

3. **jsonwebtoken (JWT)**
→ Used to create and verify **tokens** for user authentication.
◆ Example: When a user logs in, you issue a token to identify them.
4. **bcrypt**
→ Used for **hashing passwords** before storing them in the database.
◆ Example: Encrypt password during signup, compare during login.

Middleware & Communication

5. **cors (Cross-Origin Resource Sharing)**
→ Allows frontend (React) to communicate with backend (Express) from different origins/ports.
◆ Example: Your React app runs on port 3000, backend on 5000 — CORS makes it work.
6. **body-parser**
→ Parses incoming request bodies (like form data or JSON).
◆ Example: req.body works when you send POST data from frontend.
⚠ Note: In newer versions of Express, express.json() and express.urlencoded() can replace it.

7. **dotenv**
→ Loads environment variables from a .env file.
 - ◆ Example: Store your MongoDB URL, JWT secret, Stripe key securely.
- 8.

File Handling & Validation

8. **multer**
→ Used to handle **file uploads** (like profile pictures, documents, etc).
 - ◆ Example: Upload product images in an e-commerce app.
9. **validator**
→ Used to validate input data like emails, phone numbers, URLs, etc.
 - ◆ Example: validator.isEmail(req.body.email)

Payments

10. **stripe / razorpay**
→ Used for **payment integration**.
 - ◆ Stripe is popular internationally; Razorpay is more common in India.
 - ◆ Helps create payment orders, verify payments, etc.

Development Utility

11. **nodemon**
→ Automatically restarts your backend server whenever you make code changes.
 - ◆ Saves time during development.

- in package.json -> "scripts": "nodemon server.js"

- Make config Folder – db.js
- Make Models, controllers, routes, uploads Folder
 - Controllers – for keeping the backend functionality
 - Routes – do just routing
 - Models – here we gonna make db model like userModel, foodModel, etc
 - Uploads – this is used to keep the data online and offline ..ie from backend to frontend and vice versa

Server.js

```
import express from "express";
import cors from "cors";
import { connectDB } from "../config/db.js";
import foodRouter from "../routes/foodRoute.js";
import userRouter from "../routes/userRoute.js";
import 'dotenv/config'
import cartRouter from "../routes/cartRoute.js";
import orderRouter from "../routes/orderRoute.js";

// app config
const app = express();
const port = 4000;

// middleware
app.use(express.json());
app.use(cors());

// db connection
connectDB();

//api endpoint
app.use("/api/food", foodRouter)
app.use("/images", express.static('uploads'))
app.use("/api/user", userRouter); // For user Auth
app.use("/api/cart", cartRouter); // For user Auth
app.use("/api/order", orderRouter);
```

```
app.get("/", (req, res) => {
  res.send("API Working");
});

// run express server
app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});
```

- `express` → Creates the backend server and handles routes.
- `cors` → Allows requests from your React frontend (Cross-Origin).
- `connectDB` → Function to connect your app to the MongoDB database.
- `foodRouter`, `userRouter`, `cartRouter`, `orderRouter` → Route files handling specific API endpoints.
- `'dotenv/config'` → Automatically loads environment variables from `.env` file.

⚙️ App Configuration

```
js
const app = express();
const port = 4000;
```

Creates an Express app and sets the server port to 4000.

🧩 Middlewares

```
js
app.use(express.json());
app.use(cors());
```

- `express.json()` → Parses incoming JSON requests (like POST data).
- `cors()` → Enables cross-origin communication between frontend and backend.

🌐 API Endpoints

```
js
app.use("/api/food", foodRouter);
app.use("/images", express.static('uploads'));
app.use("/api/user", userRouter);
app.use("/api/cart", cartRouter);
app.use("/api/order", orderRouter);
```

- `/api/food` → Handles food-related operations (CRUD).
- `/images` → Serves uploaded images from the `uploads` folder.
- `/api/user` → Handles user authentication and profile operations.
- `/api/cart` → Manages user's cart actions.
- `/api/order` → Manages order placement and tracking.

🕒 Test Route

```
js
app.get("/", (req, res) => {
  res.send("API Working");
});
```

Simple GET route to check if the backend is running.

🚀 Start Server

```
js
app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});
```

Starts the server on port 4000 and logs the URL.

🗄️ Database Connection

```
js
connectDB();
```

Connects to MongoDB using Mongoose.

- MongoDB is setuped – new cluster and copy the string from the driver and connect
- Is u loose or exposed a key just change the db password all sorted!!
- Store the url in .env file!!

Db.js file

```
import mongoose from "mongoose";
import dotenv from "dotenv";

dotenv.config();

export const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("✅ DB Connected");
  } catch (err) {
    console.error("❌ DB Connection Failed:", err);
  }
};
```

One Model eg – orderModel.js

```
import mongoose from "mongoose";

const orderSchema = new mongoose.Schema({
  userId: { type: String, required: true },
  items: { type: Array, required: true },
  amount: { type: Number, required: true },
  address: { type: Object, required: true },
  status: { type: String, default: "Food Processing" },
  date: { type: Date, default: Date.now },
  payment: { type: Boolean, default: false },
});

const orderModel = mongoose.models.order || mongoose.model("order", orderSchema);

export default orderModel;
```

One Route eg – orderRoute.js

```
import express from "express";
import { createOrder, verifyPayment, getOrdersByUser, listOrders } from "../controllers/orderController.js";
import authMiddleware from "../middleware/auth.js";

const orderRouter = express.Router();

// Create order (requires authentication)
orderRouter.post("/create", authMiddleware, createOrder);

// Verify payment (no auth needed)
orderRouter.post("/verify", verifyPayment);

// Get user orders (for future use)
orderRouter.get("/userorders", authMiddleware, getOrdersByUser);

// Admin: Get all orders
orderRouter.get("/list", listOrders);
```

```
export default orderRouter;
```

One Controller eg - orderController.js

```
import Razorpay from "razorpay";
import crypto from "crypto";
import orderModel from "../models/orderModel.js";

// ✅ Initialize Razorpay
const razorpay = new Razorpay({
  key_id: process.env.RAZORPAY_KEY_ID,
  key_secret: process.env.RAZORPAY_KEY_SECRET,
});

// ✅ Create order
export const createOrder = async (req, res) => {
  try {
    const { items, amount, address } = req.body;

    // Get userId from auth middleware (stored in req.userId)
    const userId = req.userId;

    if (!userId) {
      return res.status(401).json({ success: false, message: "User authentication required" });
    }

    if (!items || items.length === 0) {
      return res.status(400).json({ success: false, message: "Cart is empty" });
    }

    // ✅ Step 1: Convert USD → INR
    const usdToInrRate = 83; // or fetch dynamically (see below)
    const convertedAmount = amount * usdToInrRate;

    // ✅ Step 2: Create Razorpay order
    const options = {
      amount: Math.round(convertedAmount * 100), // ₹ → paise
      currency: "INR",
      receipt: `receipt_${Date.now()}`,
    };

    const razorpayOrder = await razorpay.orders.create(options);

    // ✅ Save order in database (payment pending)
    const newOrder = new orderModel({
      userId,
      items,
      amount,
      address,
      status: "Food Processing",
      payment: false,
    });

    await newOrder.save();

    // ✅ Send response to frontend
    res.json({
      success: true,
      key: process.env.RAZORPAY_KEY_ID,
      amount: razorpayOrder.amount,
      currency: razorpayOrder.currency,
      orderId: razorpayOrder.id, // Razorpay order ID
      newOrderId: newOrder._id, // MongoDB order ID
    });
  }
}
```

```

    } catch (error) {
      console.error("Error creating order:", error);
      res.status(500).json({ success: false, message: "Error creating order" });
    }
  };

// ✅ Verify payment
export const verifyPayment = async (req, res) => {
  try {
    const { razorpay_order_id, razorpay_payment_id, razorpay_signature, orderId } = req.body;

    if (!razorpay_order_id || !razorpay_payment_id || !razorpay_signature || !orderId) {
      return res.status(400).json({ success: false, message: "Missing payment details" });
    }

    // ✅ Verify signature
    const body = razorpay_order_id + "|" + razorpay_payment_id;
    const expectedSignature = crypto
      .createHmac("sha256", process.env.RAZORPAY_KEY_SECRET)
      .update(body.toString())
      .digest("hex");

    if (expectedSignature === razorpay_signature) {
      // ✅ Payment verified successfully
      await orderModel.findByIdAndUpdate(orderId, {
        payment: true,
        status: "Order Confirmed"
      });

      res.json({ success: true, message: "Payment verified successfully" });
    } else {
      res.status(400).json({ success: false, message: "Invalid payment signature" });
    }
  } catch (error) {
    console.error("Error verifying payment:", error);
    res.status(500).json({ success: false, message: "Payment verification failed" });
  }
};

// ✅ Get orders by user (optional - for future MyOrders page)
export const getOrdersByUser = async (req, res) => {
  try {
    const userId = req.userId; // From auth middleware
    const orders = await orderModel.find({ userId }).sort({ date: -1 });
    res.json({ success: true, orders });
  } catch (error) {
    console.error("Error fetching orders:", error);
    res.status(500).json({ success: false, message: "Error fetching orders" });
  }
};

// ✅ List all orders (Admin)
export const listOrders = async (req, res) => {
  try {
    const orders = await orderModel.find()
      .populate("userId", "name email") // optional: show user info
      .sort({ createdAt: -1 }); // latest orders first

    res.json({ success: true, orders });
  } catch (error) {
    console.error("Error fetching orders:", error);
    res.status(500).json({ success: false, message: "Failed to fetch orders" });
  }
};

```

- Before implementing to the web page – do test it out on the postman or thunderclient in vs code

Admin

- Install same as we did for frontend folder
- In app.js setup same routing for list, add and order pages

List - list all the items present in the sb

Add - this will allow to add new items in the db

Order - this will list all the order that evry user created

- Navbar and sidebardone hai ye sab to!

For NOTIFICATION --- use toastify for this

For using these fuction of controller we have to use Axios -- this will be like we doing post, get, delete

Now

- User Auth is need and see the diff btw Authetication and Authoricization
- Add btn in navbar for logout and view my orders.

Now this is the whole project

- **Make more APIs and integrate with the frontend**

Most imp File – storeContext.js

```
import { createContext, useEffect, useState } from "react";
import axios from "axios";
export const StoreContext = createContext(null);

const StoreContextProvider = (props) => {
  const [cartItems, setCartItems] = useState({});
  const url = "http://localhost:4000";
  const [token, setToken] = useState(null); // ✅ null initially (not empty string)
  const [food_list, setFoodList] = useState([]);

  // -----
  // ! Fetch Food List
  // -----
  const fetchFoodList = async () => {
    try {
      const response = await axios.get(`${url}/api/food/list`);
      if (response.data.success) {
        setFoodList(response.data.data);
      } else {
        console.error("Failed to load food list:", response.data.message);
      }
    } catch (error) {
      console.error("Error fetching food list:", error);
    }
  };

  // -----
  // ! Fetch Cart Data
  // -----
  const fetchCart = async (savedToken) => {
    try {
      const res = await axios.get(`${url}/api/cart/get`, {
```



```

    headers: { token: savedToken || token },
  });
  if (res.data.success) {
    setCartItems(res.data.cartData);
  }
} catch (error) {
  console.error("Error fetching cart:", error);
}
};

// -----
// ! Load Token + Fetch Initial Data
// -----
useEffect(() => {
  const savedToken = localStorage.getItem("token");
  if (savedToken) {
    setToken(savedToken);
    fetchCart(savedToken);
  } else {
    setToken(""); // explicitly mark as logged-out
  }
  fetchFoodList();
}, []);

// -----
// ! Keep Token in LocalStorage
// -----
useEffect(() => {
  if (token) {
    localStorage.setItem("token", token);
  } else {
    localStorage.removeItem("token");
  }
}, [token]);

// -----
// ! Cart Handling
// -----
const addToCart = async (itemId) => {
  if (!cartItems[itemId]) {
    setCartItems((prev) => ({ ...prev, [itemId]: 1 }));
  } else {
    setCartItems((prev) => ({ ...prev, [itemId]: prev[itemId] + 1 }));
  }
  if (token) {
    await axios.post(`${url}/api/cart/add`, { itemId }, { headers: { token } });
  }
};

const removeFromCart = async (itemId) => {
  setCartItems((prev) => ({ ...prev, [itemId]: prev[itemId] - 1 }));

  if (token) {
    await axios.post(url + "/api/cart/remove", { itemId }, { headers: { token } });
  }
};

const getTotalCartAmount = () => {
  let totalAmount = 0;
  for (const item in cartItems) {
    if (cartItems[item] > 0) {
      const itemInfo = food_list.find((product) => product._id === item);
      if (itemInfo) totalAmount += itemInfo.price * cartItems[item];
    }
  }
  return totalAmount;
};

// -----
// ! Clear Cart after Order
// -----
const clearCart = async () => {
  try {
    if (token) {

```

```

    await axios.post(`${url}/api/cart/clear`, {}, { headers: { token } }); // ✅ ask backend to clear
  }
} catch (error) {
  console.error("Error clearing cart in backend:", error);
}

// ✅ clear frontend cart state
setCartItems({});
};

// -----
// ! Logout
// -----
const logout = () => {
  setToken("");
  setCartItems({});
  localStorage.removeItem("token");
};

const contextValue = {
  food_list,
  cartItems,
  setCartItems,
  addToCart,
  removeFromCart,
  getTotalCartAmount,
  url,
  token,
  setToken,
  fetchFoodList,
  logout,
  clearCart,
};

// ✅ Don't render until token state is restored
if (token === null) return null;

return (
  <StoreContext.Provider value={contextValue}>
    {props.children}
  </StoreContext.Provider>
);
};

export default StoreContextProvider;

```

END